

```
In [2]: import pandas as pd
import numpy as np
import tensorflow as tf
```

```
In [3]: df = pd.read_csv(r'C:\Users\sande\OneDrive\Desktop\Naresh IT\data science and ai\Ar
```

```
In [4]: df.head(4)
```

```
Out[4]:
```

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	E
0	1	15634602	Hargrave	619	France	Female	42	2	
1	2	15647311	Hill	608	Spain	Female	41	1	85
2	3	15619304	Onio	502	France	Female	42	8	159
3	4	15701354	Boni	699	France	Female	39	1	



```
In [5]: X = df.iloc[:,3:-1].values
y = df.iloc[:, -1].values
```

```
In [6]: X
```

```
Out[6]: array([[619, 'France', 'Female', ..., 1, 1, 101348.88],
               [608, 'Spain', 'Female', ..., 0, 1, 112542.58],
               [502, 'France', 'Female', ..., 1, 0, 113931.57],
               ...,
               [709, 'France', 'Female', ..., 0, 1, 42085.58],
               [772, 'Germany', 'Male', ..., 1, 0, 92888.52],
               [792, 'France', 'Female', ..., 1, 0, 38190.78]], dtype=object)
```

```
In [7]: from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
X[:,2] = le.fit_transform(X[:,2])
```

```
In [8]: X
```

```
Out[8]: array([[619, 'France', 0, ..., 1, 1, 101348.88],
               [608, 'Spain', 0, ..., 0, 1, 112542.58],
               [502, 'France', 0, ..., 1, 0, 113931.57],
               ...,
               [709, 'France', 0, ..., 0, 1, 42085.58],
               [772, 'Germany', 1, ..., 1, 0, 92888.52],
               [792, 'France', 0, ..., 1, 0, 38190.78]], dtype=object)
```

```
In [9]: from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder

ct = ColumnTransformer(transformers=[('encoder', OneHotEncoder(), [1])], remainder='passthrough')
X = np.array(ct.fit_transform(X))
```

```
In [10]: from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X = sc.fit_transform(X)
```

```
In [11]: X
```

```
Out[11]: array([[ 0.99720391, -0.57873591, -0.57380915, ...,  0.64609167,
                  0.97024255,  0.02188649],
                [-1.00280393, -0.57873591,  1.74273971, ..., -1.54776799,
                  0.97024255,  0.21653375],
                [ 0.99720391, -0.57873591, -0.57380915, ...,  0.64609167,
                 -1.03067011,  0.2406869 ],
                ...,
                [ 0.99720391, -0.57873591, -0.57380915, ..., -1.54776799,
                  0.97024255, -1.00864308],
                [-1.00280393,  1.72790383, -0.57380915, ...,  0.64609167,
                 -1.03067011, -0.12523071],
                [ 0.99720391, -0.57873591, -0.57380915, ...,  0.64609167,
                 -1.03067011, -1.07636976]])
```

```
In [12]: from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size = 0.20, random_state
```

```
In [13]: ann = tf.keras.models.Sequential()

ann.add(tf.keras.layers.Dense(units = 6,activation = 'relu'))
ann.add(tf.keras.layers.Dense(units = 6,activation = 'relu'))
ann.add(tf.keras.layers.Dense(units = 5,activation = 'relu'))
ann.add(tf.keras.layers.Dense(units = 4,activation = 'relu'))
ann.add(tf.keras.layers.Dense(units = 1,activation = 'sigmoid'))
```

```
In [14]: ann.compile(optimizer = 'adam',loss = 'binary_crossentropy',metrics = ['accuracy'])
```

```
In [15]: ann.fit(X_train,y_train,batch_size = 32,epochs = 20,validation_data=(X_test,y_test))
```

Epoch 1/20
250/250 ————— 2s 3ms/step - accuracy: 0.6205 - loss: 0.7011 - val_accuracy: 0.7985 - val_loss: 0.4864
Epoch 2/20
250/250 ————— 1s 2ms/step - accuracy: 0.8027 - loss: 0.4752 - val_accuracy: 0.8080 - val_loss: 0.4585
Epoch 3/20
250/250 ————— 1s 2ms/step - accuracy: 0.8090 - loss: 0.4569 - val_accuracy: 0.8080 - val_loss: 0.4396
Epoch 4/20
250/250 ————— 1s 2ms/step - accuracy: 0.8110 - loss: 0.4308 - val_accuracy: 0.8180 - val_loss: 0.4059
Epoch 5/20
250/250 ————— 1s 2ms/step - accuracy: 0.8244 - loss: 0.4074 - val_accuracy: 0.8290 - val_loss: 0.3839
Epoch 6/20
250/250 ————— 1s 2ms/step - accuracy: 0.8294 - loss: 0.3925 - val_accuracy: 0.8355 - val_loss: 0.3754
Epoch 7/20
250/250 ————— 1s 2ms/step - accuracy: 0.8355 - loss: 0.3825 - val_accuracy: 0.8445 - val_loss: 0.3647
Epoch 8/20
250/250 ————— 1s 3ms/step - accuracy: 0.8393 - loss: 0.3768 - val_accuracy: 0.8470 - val_loss: 0.3614
Epoch 9/20
250/250 ————— 1s 3ms/step - accuracy: 0.8435 - loss: 0.3717 - val_accuracy: 0.8505 - val_loss: 0.3568
Epoch 10/20
250/250 ————— 1s 2ms/step - accuracy: 0.8456 - loss: 0.3684 - val_accuracy: 0.8510 - val_loss: 0.3560
Epoch 11/20
250/250 ————— 1s 3ms/step - accuracy: 0.8474 - loss: 0.3653 - val_accuracy: 0.8545 - val_loss: 0.3516
Epoch 12/20
250/250 ————— 1s 3ms/step - accuracy: 0.8485 - loss: 0.3626 - val_accuracy: 0.8540 - val_loss: 0.3539
Epoch 13/20
250/250 ————— 1s 3ms/step - accuracy: 0.8506 - loss: 0.3621 - val_accuracy: 0.8545 - val_loss: 0.3505
Epoch 14/20
250/250 ————— 1s 3ms/step - accuracy: 0.8495 - loss: 0.3604 - val_accuracy: 0.8540 - val_loss: 0.3483
Epoch 15/20
250/250 ————— 1s 2ms/step - accuracy: 0.8484 - loss: 0.3594 - val_accuracy: 0.8520 - val_loss: 0.3492
Epoch 16/20
250/250 ————— 1s 2ms/step - accuracy: 0.8528 - loss: 0.3568 - val_accuracy: 0.8610 - val_loss: 0.3446
Epoch 17/20
250/250 ————— 1s 3ms/step - accuracy: 0.8518 - loss: 0.3573 - val_accuracy: 0.8580 - val_loss: 0.3452
Epoch 18/20
250/250 ————— 1s 3ms/step - accuracy: 0.8535 - loss: 0.3555 - val_accuracy: 0.8575 - val_loss: 0.3451
Epoch 19/20
250/250 ————— 1s 2ms/step - accuracy: 0.8535 - loss: 0.3553 - val_accuracy:

uracy: 0.8615 - val_loss: 0.3441

Epoch 20/20

250/250 ————— 1s 3ms/step - accuracy: 0.8522 - loss: 0.3541 - val_acc

uracy: 0.8550 - val_loss: 0.3451

Out[15]: <keras.src.callbacks.history.History at 0x1c87bd589b0>

In [16]: `y_pred = ann.predict(X_test)`

63/63 ————— 0s 2ms/step

In [17]: `y_pred = (y_pred > 0.5)`

In [18]: `print(np.concatenate((y_pred.reshape(len(y_pred),1),y_test.reshape(len(y_test),1)),`

```
[[0 0]
 [0 1]
 [0 0]
 ...
 [0 0]
 [0 0]
 [0 0]]
```

In [19]: `from sklearn.metrics import accuracy_score, confusion_matrix`

```
ac = accuracy_score(y_test,y_pred)
ac
```

Out[19]: 0.855

In [20]: `cm = confusion_matrix(y_test,y_pred)`
`cm`

Out[20]: `array([[1505, 90],
 [200, 205]], dtype=int64)`